



MARATONA DE PROGRAMAÇÃO

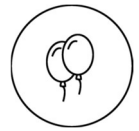
CADERNO DE QUESTÕES

Edição 2024



INSTITUTO FEDERAL
Sul-rio-grandense

Campus
Charqueadas



Instruções

LEIA ATENTAMENTE ESTAS INSTRUÇÕES ANTES DE INICIAR A PROVA

Este caderno de tarefas é composto por 10 questões. Verifique se o caderno está completo.

A prova deve ser feita em conjunto apenas pelos membros da equipe.

Deixe seu celular sobre a bancada e não use o mesmo. Cada equipe pode usar apenas seu computador previamente alocado e mais nenhum recurso computacional.

É proibido consultar a Internet ou acessar o celular durante a prova, bem como utilizar de plugins que utilizem de recursos de inteligência artificial para resolução dos exercícios, sob pena de eliminação de toda a equipe.

É permitido consultar qualquer material físico impresso ou manuscrito que a equipe tenha trazido consigo para a prova.

É permitida a consulta ao help do ambiente de programação, se este estiver disponível.

Os problemas têm o mesmo valor na correção.

A correção é automatizada, portanto siga atentamente às exigências da tarefa quanto ao formato da entrada e saída de seu programa.

Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.

Os problemas possuem níveis de dificuldade diversos, de fácil a difícil, ordenadas aleatoriamente; cabe à equipe identificá-los e montar a estratégia mais conveniente.

Preste muita atenção no nome dos arquivos fonte que vocês escolherem: **Não usem caracteres especiais ou espaços em branco no nome do arquivo.**

Soluções na linguagem C devem ser arquivos extensão .c; soluções na linguagem C++ devem ser arquivos com extensão .cpp; soluções na linguagem PHP devem ser arquivos com extensão .php; soluções na linguagem Java devem ser arquivos com extensão .java e a classe principal deve ter o mesmo nome do arquivo fonte; soluções em Python2 ou Python3 devem ter a extensão .py ou .py3, respectivamente.

Para problemas diferentes a equipe pode escolher trabalhar com linguagens diferentes; até que a solução para um problema seja aceita, linguagens diferentes podem ser usadas nas submissões de cada problema.

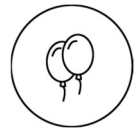
As soluções devem ser enviadas para correção através do formulário disponível no autojudge (192.168.208.192:3000). Cada submissão errada é penalizada em 20 minutos a mais no tempo total de atividade da equipe; entretanto, o tempo gasto para a solução da questão só é contabilizado uma vez que uma solução é aceita.

Lembre-se: ganha quem resolver mais questões. O tempo é critério de desempate.

Não se esqueça, em maratonas de programação usamos um ambiente automatizado para a avaliação do código (por exemplo, aqui usaremos o autojudge). Assim, você pode fazer a entrada e saída de dados como desejar durante os testes de seu código mas na submissão do exercício para o sistema **atente-se para o padrão utilizado no exemplo presente nas páginas seguintes (adaptados para cada linguagem)**. O período inicial da competição será reservado para executar um teste simples de entrada e saída de dados com as funcionalidades de entrada e saída usadas como padrão para o autojudge.

O mais importante de TUDO: divirtam-se programando!

Boa maratona!



Instruções para novatos em maratonas de programação

SE É SUA PRIMEIRA VEZ EM MARATONAS DE PROGRAMAÇÃO DE EVENTOS DE MARATONA ESTA SEÇÃO PODE SER ÚTIL

Como você leu nas instruções, você usará na maratona comandos de entrada e saída diferentes do que costuma usar em aula. Ocorre que nos exercícios de nossas aulas uma pessoa testa o código, interpretando o papel de usuário de seu programa (essa pessoa pode ser você ou o professor). Assim, nos exercícios em aula usamos comandos de entrada e saída típicos para interação com um usuário, que se refletem em comandos de leitura por meio de teclado/mouse (por exemplo `window.prompt` e `window.confirm` quando trabalhamos com o Javascript básico) ou comandos de escrita para mostrar informações para o usuário no monitor (também exemplificando por meio do javascript, usamos comandos para interagir pelo navegador, como `window.alert` ou `document.write`).

Já em uma maratona, o competidor submete suas respostas para que um sistema automatizado (em nosso caso `autojudge`) interaja com o programa resultante. Assim, tomando como exemplo um programa criado em Javascript, o `autojudge` não interage por meio de um teclado ou mesmo navegador e sim pela comunicação por meio de arquivos de texto. Portanto, precisamos adaptar os comandos de entrada e saída para comandos que possam ser utilizados pelo sistema, realizando a leitura e escrita de arquivos de texto.

Por exemplo, um código Javascript que pretende ler dois valores A e B e escrever a soma dos mesmos poderia ser feito por um estudante desta forma na disciplina de lógica:

```
let a = Number(prompt('Digite um valor numérico para A'));
let b = Number(prompt('Digite um valor numérico para B'));
let c = a+b;
alert(c);
```

Já se estamos criando código para o `autojudge` avaliar, podemos assumir que ele enviará como entrada para nosso código um arquivo de texto com todos os inputs que serão utilizados (no caso os valores de a e b), e depois comparará o resultado com um arquivo de saída que ele possui. Suponha por exemplo, que o `autojudge` vai testar este programa com um arquivo de entrada que possui o seguinte conteúdo

3

2

(a primeira linha do arquivo possui o valor 3 e a segunda possui o valor 2, os quais devem ser atribuídos respectivamente para a e b)

Assim, neste mesmo exemplo de soma de dois valores, teríamos o seguinte código (comentado)

```
let input = require('fs').readFileSync('/dev/stdin', 'utf8');    //aqui vamos deixar o programa apto a receber do
                                                                //autojudge um arquivo com os dados de entrada

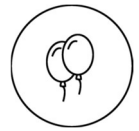
let lines = input.split('\n'); //”quebramos” o arquivo em linhas diferentes. Essas linhas estão armazenadas na variável
                               // lines, que é numerada iniciando em 0

let a = parseInt(lines[0]); //pegamos o conteúdo da primeira linha do arquivo (3) - podemos usar Number ao invés de
                             //parseInt, entretanto parseInt garante que o numero será um inteiro

let b = parseInt(lines[1]); //pegamos o conteúdo da segunda linha do arquivo (2)

let c = a+b;                //somamos os dois valores (usual, como o exemplo anterior)

console.log(c)              //apresentamos o resultado, no caso do autojudge, por meio do comando console.log
```



Perceba que a diferença nos códigos está nos comandos de leitura e escrita (entrada e saída) usados na linguagem. Todo o programa (que neste exemplo se traduz na soma de dois valores) permanecerá igual. Assim, você pode fazer o seu programa usando os comandos de entrada e saída que costuma usar em aula, alterando estes comandos apenas no momento em que quiser submeter o exercício, ou criar um arquivo texto e usar a mesma estrutura que o autojudge, simplesmente adicionando ao `readFileSync` o caminho para o seu arquivo de teste.

Em um segundo exemplo de exercício, temos um código que deve ler um arquivo, o qual possui a quantidade N de somas que devem ser feitas e, nas linhas seguintes, as somas que devem ser realizadas, apresentando o resultado de cada uma delas. Se estivéssemos resolvendo este exercício em uma aula, teríamos um código neste formato:

```
let n = Number(prompt("informe quantas somas deverão ser realizadas"));
for(let i=0; i<n;i++){
    let a = Number(prompt('Digite um valor numérico para A'));
    let b = Number(prompt('Digite um valor numérico para B'));
    let c = a+b;
    alert(c);
}
```

Já para uma maratona, teremos por exemplo o arquivo de entrada:

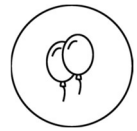
```
3
2 3
5 2
3 5
```

(note que o sistema esperará a geração de uma saída com os valores: 5 7 8, o resultado das somas em cada linha)

Um código resultante possível seria:

```
let input = require('fs').readFileSync('/dev/stdin', 'utf8');
let lines = input.split('\n');
let n = parseInt(input[0]);
for(let i=1; i<=n; i++){
    let numbers = input[i].split(' ');
    let a = numbers[0];
    let b = numbers[1];
    let c = a+b;
    console.log(c);
}
```

A seguir teremos exemplos de um exercício simples de soma nas diferentes linguagens aceitas pelo autojudge.



Exemplo de exercício

Leia 2 valores inteiros e armazene-os nas variáveis A e B. Efetue a soma de A e B atribuindo o seu resultado na variável X. Imprima X conforme exemplo apresentado abaixo. Não apresente mensagem alguma além daquilo que está sendo especificado

Entrada

A entrada contém 2 valores inteiros.

Saída

Imprima a mensagem "X = " (letra X maiúscula) seguido pelo valor da variável X e pelo final de linha. Cuide para que tenha um espaço antes e depois do sinal de igualdade, conforme o exemplo abaixo.

Exemplo de código em diferentes linguagens

C++

```
#include <iostream>
using namespace std;
int main() {
    int A, B, X;
    cin >> A >> B;
    X = A + B;
    cout << "X = " << X << endl;
    return 0;
}
```

C

```
#include <stdio.h>
int main() {
    int A, B, X;
    scanf("%d", &A);
    scanf("%d", &B);
    X=A+B;
    printf("X = %d\n", X);
    return 0;
}
```

PHP

```
<?php
$a = trim(fgets(STDIN));
$b = trim(fgets(STDIN));
$soma = $a + $b;
echo "X = " . $soma . "n";
?>
```

Python

```
a = int(input())
b = int(input())
x = a + b
print('X =', x)
```

Java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class Main {
    public static void main(String[] args) throws
    IOException {
        InputStreamReader ir = new InputStreamReader(
        System.in);
        BufferedReader in = new BufferedReader(ir);
        int A, B, X;
        A = Integer.parseInt(in.readLine());
        B = Integer.parseInt(in.readLine());
        X = A + B;
        System.out.printf("X = %d\n", X);
    }
}
```

Javascript

```
var input = require('fs').readFileSync('/dev/stdin', 'utf8');
var lines = input.split('\n');
var a = parseInt(lines.shift());
var b = parseInt(lines.shift());
console.log('X = ' + (a+b));
```



Mineração nas Profundezas

Durante uma escavação submarina, mineradores estão à procura de um mineral raro cuja concentração ideal varia conforme a profundidade. O mineral só é estável em uma concentração específica, e o objetivo dos mineradores é encontrar a profundidade em que a concentração do mineral seja o mais próxima possível da concentração ideal.

O paredão rochoso onde o mineral é encontrado tem uma distribuição de concentração que cresce com a profundidade, porém de forma variável, o que implica que a concentração do mineral não segue uma progressão linear. Para determinar a profundidade com a concentração mais próxima da desejada, o minerador utiliza um equipamento de perfuração.

Contudo, cada tentativa de perfuração é um processo dispendioso. Assim, a tarefa consiste em determinar o número mínimo de perfurações necessárias para encontrar a concentração do mineral mais próxima da ideal, minimizando o número de tentativas.

É importante observar que, em alguns casos, podem existir valores repetidos de concentração ou empates nas diferenças de proximidade com a concentração ideal. Caso isso ocorra, deve-se escolher a profundidade mais rasa (menor número de profundidade) onde a concentração mais próxima foi encontrada.

Entrada

A primeira linha contém n inteiros, separados por espaços, representando as concentrações (em kg/m^3) do mineral a cada metro de profundidade. A profundidade começa em 0 e segue até $n-1$ metros.

A segunda linha contém um único número inteiro, representando a concentração ideal (em kg/m^3) do mineral.

Saída

O programa deve imprimir três números, separados por um espaço:

1. O número mínimo de tentativas de perfuração necessárias para encontrar o mineral em sua concentração mais próxima da ideal.
2. A concentração do mineral encontrada, que será a mais próxima possível da ideal.
3. A profundidade onde esta concentração foi encontrada, considerando que a profundidade inicial é 0.

Entrada	Saída
50 100 150 200 250 300 350 400 450 500 270	3 250 5
50 100 100 200 250 300 350 400 450 500 150	4 100 1
10 20 20 20 30 30 30 40 40 50 35	4 30 4



<Página intencionalmente deixada em branco>

O catálogo de receitas

Rodolfo é um cozinheiro talentoso e apaixonado pela gastronomia. Ele está criando um sistema para catalogar todas as suas receitas e facilitar o acesso a elas. Cada receita é identificada por um padrão único, que é gerado com base no nome completo da receita e no ano em que foi criada. Esse identificador é formado pelas primeiras letras de cada palavra no nome da receita, seguidas do ano em que a receita foi criada.



Por exemplo, se uma receita chamada “Bolo de Cenoura” foi criada em 2023, o identificador gerado seria “bdc2023”.

No entanto, Rodolfo percebeu que, ao longo do tempo, algumas receitas podem ter nomes semelhantes, gerando duplicidade nos identificadores. Quando isso acontece, ele precisa dar nomes diferentes para essas receitas, mas sem perder o padrão de identificação.

Rodolfo precisa saber quantas receitas precisarão de um identificador fora do padrão para que ele possa ajustar o sistema e evitar conflitos de identificação.

Objetivo

Desenvolver um programa que ajude Rodolfo a calcular o número de receitas que precisarão de ajuste no nome.

Entrada

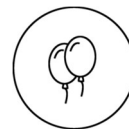
A primeira linha contém dois números inteiros N ($1 \leq N \leq 100$) e A ($1 \leq A \leq 9999$), representando o número de receitas e o ano atual, respectivamente.

As N linhas seguintes contêm os nomes das receitas, as quais são representadas com até 100 caracteres contendo apenas espaços e letras minúsculas com pelo menos uma letra.

Saída

Um número inteiro que corresponde ao número de receitas que receberão um identificador fora do padrão.

Entrada	Saída
4 2024 bolo de cenoura torta de figo com nozes bombom de caramelo alcachofra ao vinho	1



6 2022 mousse de chocolate bolo de milho verde biscoito de milho moqueca de corvina brownie de mascarpone risoto de cogumelos	2
3 2023 gelato de pistache escondidinho de carne seca esfiha de carne	0



<Página intencionalmente deixada em branco>

Contador de Calorias

Para manter o metabolismo acelerado pelo tempo máximo, os nutricionistas recomendam que se realize uma refeição a cada 2-3 horas. O fracionamento de refeições geralmente resulta em até 7 (sete) pequenas refeições por dia.

Aplicativos como MyFitnessPal, LifeSum e FatSecret, permitem o registro dos macronutrientes de cada refeição. Esse registro é feito na leitura do código de barras de um produto para consulta da tabela nutricional ou consultando uma base de dados com os macronutrientes de determinado alimento. Ao final do dia, o aplicativo disponibiliza um relatório completo sobre as calorias que o usuário ingeriu naquele dia.

	Matriz M			Vetor Y
	carbo (g)	proteina (g)	gordura (g)	cal por refeicao
cafe da manha	80	20	10	490
lancha da manha	20	10	5	165
almoco	100	30	15	655
lanche da tarde	20	10	5	165
janta	100	30	15	655
lanche da noite	20	10	5	165
ceia	5	25	10	210
Vetor X – cal por macronutriente				Total = T = 2505
	1380	540	585	

Considere que o registro das refeições diárias é realizado em uma matriz de 7x3, onde cada linha representa uma refeição e cada coluna representa um macronutriente, nesta ordem: (i) carboidrato, (ii) proteína e (iii) gordura. Além disso, sabe-se que 1 grama de gordura corresponde a 9 calorias, 1 grama de proteína e carboidrato correspondem a 4 calorias.

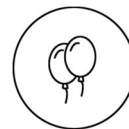
Neste tipo de aplicativo, é primordial extrair (i) quantas calorias foram consumidas por refeição, (ii) quantas calorias foram consumidas por macronutriente e (iii) quantas calorias foram consumidas no total. Considerando o exemplo da figura, a matriz M contém a quantidade de gramas de cada distribuída por refeição, o vetor $X = \{1380, 540, 585\}$ registra a quantidade de calorias por refeição, o vetor $Y = \{490, 165, 655, 165, 655, 165, 210\}$ registra a quantidade de calorias por macronutriente e $T = 2505$ armazena o total de calorias ingeridas no dia.

Entrada

A entrada da matriz M é descrita em sete linhas. Em cada linha i são lidos 3 números inteiros $M_{i,1}$, $M_{i,2}$ e $M_{i,3}$ ($0 < M_{i,j} \leq 1000$, para todo $M_{i,j}$, tal que $1 \leq i \leq 7$ e $1 \leq j \leq 3$) que correspondem a quantidade de gramas por macronutriente ingeridas pelo usuário. Espera-se que $M_{i,1}$, $M_{i,2}$ e $M_{i,3}$ correspondam à carboidrato, proteína e gordura, respectivamente.

Saída

A saída tem três linhas. A primeira linha corresponde ao vetor X ($0 \leq X_i \leq 27000$, para todo X_i , tal que $1 \leq i \leq 3$) contém a quantidade de calorias por macronutriente ingeridas em um dia pelo usuário. A segunda linha corresponde ao vetor Y ($0 \leq Y_i \leq 17000$, para todo Y_i , tal que $1 \leq i \leq 7$) contém a quantidade de calorias de ingeridas por refeição em um dia pelo usuário. A última linha contém T ($0 \leq T \leq 27000$) contendo o total de calorias ingeridas em um dia



Entrada	Saída
80 20 10 20 10 5 100 30 15 20 10 5 100 30 15 20 10 5 5 25 10	1380 540 585 490 165 655 165 655 165 210 2505
120 20 30 0 0 0 150 80 50 0 0 0 120 10 25 0 0 0 0 0 0	1560 440 945 830 0 1370 0 745 0 0 2945



<Página intencionalmente deixada em branco>



Alocação dos Recipientes

No coração de um laboratório de química, onde substâncias misteriosas borbulham e novas descobertas aguardam serem feitas, os cientistas enfrentam um dilema logístico. Em uma série de experimentos, eles precisam armazenar uma grande quantidade de líquidos em recipientes de vidro, de capacidades variadas. Mas, para garantir a segurança e a eficiência do processo, cada gota precisa ser alocada de forma estratégica.

Os recipientes no laboratório não são todos iguais: eles vêm em diferentes tamanhos, com capacidades que variam de pequenas doses a volumes generosos. A missão dos cientistas é otimizar o uso desses recipientes, de modo a minimizar a quantidade utilizada. Para isso, eles estabelecem uma regra de ouro: sempre usar primeiro os recipientes de maior capacidade, enchendo-os até a borda antes de recorrer aos menores. Dessa forma, evitam desperdícios e conseguem realizar os experimentos com mais segurança.

Sua tarefa é desenvolver um programa que ajude os cientistas nessa tarefa de organização, calculando o número mínimo de recipientes necessários para armazenar uma quantidade específica de líquido.

Entrada:

1. A primeira linha de entrada contém uma lista de números inteiros, representando as capacidades, em mililitros, dos recipientes de vidro disponíveis no laboratório. Os valores estão separados por espaços, e cada valor representa a capacidade de um recipiente.
2. A segunda linha de entrada contém um único número em ponto flutuante, que representa a quantidade total de líquido a ser armazenada, em litros. Para simplificar, considere que 1 litro equivale a 1000 mililitros.

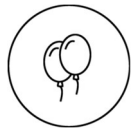
Saída:

O programa deve exibir o número mínimo de recipientes necessários para armazenar toda a quantidade de líquido especificada. Caso a quantidade de líquido ultrapasse a capacidade total dos recipientes disponíveis, o programa deve retornar -1, indicando que não é possível armazenar o volume solicitado com os recipientes à disposição.

Entrada	Saída
500 1000 200 750 1.2	2
300 600 100 200 1.5	-1
1000 550 200 25 500 630 50 100 250 150 3.2	7



<Página intencionalmente deixada em branco>



O roubo de doces



Jujuba, o maior ladrão de doces de Charqueadas, planejou o golpe da sua vida: invadir a Fábrica Delícia, que guarda os doces mais preciosos em salas secretas. A fábrica possui M salas, cada uma contendo um tipo raro de doce, como balas de caramelo salgado, pirulitos de framboesa, chocolates com folha de ouro e muito mais.

Jujuba, sem saber qual chave abre qual sala, encontrou um molho com N chaves. Cada chave abre exatamente uma sala, mas ele precisa descobrir qual chave abre qual porta no menor número de tentativas possível, antes que o alarme soe e a segurança o capture.

Como ele é apaixonado por doces e não quer perder tempo, ele vai utilizar seus conhecimentos em Informática para criar um programa que o ajude a calcular o menor número de tentativas para descobrir qual chave abre cada sala de forma eficiente.

Objetivo

Desenvolver um programa que ajude Jujuba a calcular o número mínimo de tentativas necessárias para encontrar a correspondência entre as N chaves e as M salas.

Entrada

Dois inteiros N e M , representando respectivamente o número de chaves e o número de salas. Considere que $1 \leq N \leq M \leq 1000$.

Saída

Um número inteiro correspondente ao número mínimo de tentativas que Jujuba deverá realizar para descobrir qual chave abre qual sala.

Entrada	Saída
1 2	1
7 11	49
200 452	70300



<Página intencionalmente deixada em branco>

Detecção de movimento

Você é responsável por desenvolver uma função de detecção de variações de movimento para uma câmera de segurança. O sistema recebe uma sequência de duas imagens em preto e branco, cada uma representada por uma matriz binária de pixels (valores 0 e 1). O objetivo é detectar se houve um deslocamento de movimento entre as duas imagens e, se houver, determinar a direção do movimento.

Cada célula da matriz representa um pixel, onde:

1 indica que o pixel está ativado (presença de movimento ou objeto). 0 indica que o pixel está desativado (fundo).

Especificação da função

Implemente uma função `detectarMovimento(matriz1, matriz2)` que receba duas matrizes 3×3 (`matriz1` e `matriz2`) e retorne uma string com a direção do movimento entre elas, se houver. As direções possíveis são:

"direita" se a imagem em `matriz2` é uma versão deslocada de `matriz1` para a direita.

"esquerda" se a imagem em `matriz2` é uma versão deslocada de `matriz1` para a esquerda.

"baixo" se a imagem em `matriz2` é uma versão deslocada de `matriz1` para baixo. "cima" se a imagem em `matriz2` é uma versão deslocada de `matriz1` para cima. "sem movimento" se `matriz1` e `matriz2` forem idênticas.

Restrições:

As matrizes terão o mesmo tamanho 3×3 .

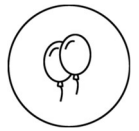
O deslocamento é de apenas uma posição em uma das quatro direções, caso exista.

Entrada:

Uma sequência de 6 linhas com 3 números (0 e 1) separados por vírgula. As primeiras 3 linhas correspondem aos valores da `matriz1`, e as demais linhas correspondem aos valores da `matriz2`.

Saída:

Uma string indicando o movimento conforme informado na especificação da função. Os valores de saída devem estar com todos os caracteres em minúsculo.



Entrada	Saída
0,0,0 0,1,0 0,0,0 0,0,0 0,0,1 0,0,0	direita
0,0,0 1,1,1 0,0,0 1,1,1 0,0,0 0,0,0	cima
0,0,0 0,0,0 1,1,1 0,0,0 0,0,1 1,1,0	cima
0,0,0 0,0,0 1,0,0 0,0,0 0,0,0 1,0,0	Sem movimento



<Página intencionalmente deixada em branco>

Camisetas da Charcode

A organização da charcode deste ano resolveu distribuir uma edição especial da camiseta do evento, apenas para campeões. Haverá um estoque limitado destas camisetas (C), que será distribuído para um número limitado de pessoas (P), onde $C \geq P$. As camisetas foram encomendadas de forma a existir a mesma quantidade de camisetas de tamanho PP, P, M, G e GG, de modo que o somatório de camisetas de cada tamanho é igual a C. (C é, necessariamente, portanto, múltiplo de 5 e cada tamanho de camiseta possui $C/5$ unidades).

Mediante estas informações, a organização do evento lhe pediu ajuda para criar um programa que verifique se, com o estoque de camisetas disponível, e caso cada campeão opte por dois tamanhos de camisetas, é possível distribuir uma camiseta de tamanho adequado para cada um deles.

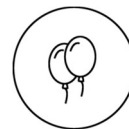
Entrada

A primeira linha do arquivo de entrada contém um número C de camisetas, onde C é múltiplo de 5 e $1 < C \leq 50$, e indica o número de camisetas disponíveis seguido de um número P de pessoas onde $1 \leq P \leq 50$, indicando o número de pessoas que devem receber a camiseta, seguido de P linhas, onde cada linha descreve as duas opções (entre PP, P, M, G e GG) de tamanhos de camisetas de cada uma das pessoas que receberá a mesma.

Saída

A palavra SIM se é possível fazer a distribuição de camisetas ou NAO se não é possível distribuir uma camiseta de tamanho adequado (entre os dois escolhidos) para cada pessoa

Entrada	Saída
20 6 G GG GG G G M M P P PP M G	SIM
5 4 P G M P G P M P	NAO
5 2 P M M P	SIM



<Página intencionalmente deixada em branco>

Mensagem Secreta

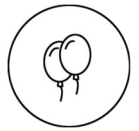
A Cifra de Vigenère é um método de criptografia que foi descrito pela primeira vez por Giovan Battista Bellaso em 1553, mas acabou sendo atribuído erroneamente a Blaise de Vigenère, que publicou um método similar em 1586. Esta cifra é uma evolução da Cifra de César e foi considerada inquebrável por mais de 300 anos.

Você foi contratado por uma agência de espionagem para desenvolver um programa que implementa a Cifra de Vigenère. Este método de criptografia utiliza uma palavra-chave para cifrar uma mensagem, tornando-a ilegível para quem não possui a chave. A Cifra de Vigenère utiliza uma tabela de 26x26 (chamada tabela de Vigenère) onde cada linha representa um alfabeto deslocado ciclicamente, conforme a tabela abaixo.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Para cifrar uma mensagem utilizando a Cifra de Vigenère:

1. Pegue a primeira letra da mensagem original (letra da mensagem) e a primeira letra da palavra-chave.
2. Localize a letra da mensagem na primeira coluna da tabela de Vigenère.
3. Localize a letra da palavra-chave na primeira linha da tabela.
4. A letra cifrada será a que está na interseção da linha da letra da mensagem com a coluna da letra da palavra-chave.
5. Repita os passos 1-4 para cada letra da mensagem, utilizando a próxima letra da palavra-chave a cada iteração. Quando a palavra-chave acabar, volte ao início.
6. Mantenha espaços e pontuação inalterados.



Por exemplo, para cifrar a palavra "ATACAR" usando a palavra-chave "LIMAO":

1. Primeira letra da mensagem: A Primeira letra da chave: L Letra cifrada: L
2. Segunda letra da mensagem: T Segunda letra da chave: I Letra cifrada: B
3. Terceira letra da mensagem: A Terceira letra da chave: M Letra cifrada: M
4. Quarta letra da mensagem: C Quarta letra da chave: A Letra cifrada: C
5. Quinta letra da mensagem: A Quinta letra da chave: O Letra cifrada: O
6. Sexta letra da mensagem: R Sexta letra da chave: L (repetindo a chave) Letra cifrada: C

Resultado final: LBMCOOC

Entrada

A primeira linha contém uma frase F ($1 \leq |F| \leq 255$) contendo apenas letras maiúsculas de A a Z (sem acentuação e sinal gráfico), espaços e pontuação. A segunda linha contém uma palavra-chave K ($2 \leq |K| \leq 10$) contendo apenas letras maiúsculas de A a Z, sem espaço e pontuação.

Saída

Uma única linha contendo a mensagem cifrada, onde apenas as letras foram modificadas segundo a Cifra de Vigenère.

Entrada	Saída
ATACAR BASE! LIMAO	LBMCOOC JMSS!
QUE SHOW DA XUXA EH ESSE! XU	NOB MEIT XX RRRX YE YPMB!
PROGRAMMING CONTEST CODE	RFRKTOPQKBJ GQBWIUH

Explicação detalhada do primeiro exemplo usando a tabela:

Mensagem: ATACAR BASE

Chave: LIMAO LIMA (chave repetida)

Para cada letra:

1. A (mensagem) + L (chave):
 - Coluna A, linha L $\rightarrow$ L
2. T (mensagem) + I (chave):
 - Coluna T, linha I $\rightarrow$ B
3. A (mensagem) + M (chave):
 - Coluna A, linha M $\rightarrow$ M
4. C (mensagem) + A (chave):
 - Coluna C, linha A $\rightarrow$ C



5. A (mensagem) + O (chave):
 - Coluna A, linha O → O
6. R (mensagem) + L (chave):
 - Coluna R, linha L → C
7. B (mensagem) + I (chave):
 - Coluna B, linha I → J
8. A (mensagem) + M (chave):
 - Coluna A, linha M → M
9. S (mensagem) + A (chave):
 - Coluna S, linha A → S
10. E (mensagem) + O (chave):
 - Coluna E, linha O → S

Resultado: LBMCOO JMSS!



<Página intencionalmente deixada em branco>

Rústica

Anualmente, o Câmpus Charqueadas organiza uma Rústica, uma corrida pelas ruas da cidade com um total de 4 km. No entanto, tem-se uma dificuldade na exibição do tempo que o competidor campeão levou para realizar a prova. Essencialmente, cada competidor recebe um identificador numérico para fixar na camiseta. Junto com esse identificador, vai um tag com um RFID embarcado que, quando o competidor passa pela linha de chegada, envia o horário no formato de um Unix Timestamp.

O Unix Timestamp é uma forma de representar a data e hora como um único número inteiro, contando o número de segundos decorridos desde 1º de janeiro de 1970, às 00:00:00 UTC (Coordinated Universal Time). Por exemplo, o Unix Timestamp para 12 de novembro de 2024 às 09:15:53 UTC é 1731413753, ou seja, se passaram 1.731.413.753 segundos desde 1º de janeiro de 1970, às 00:00:00 UTC até esse momento.

Sua tarefa é criar um programa que receba as informações sobre a Rústica do Câmpus Charqueadas e determine quem foi o primeiro colocado, informando o tempo da corrida no formato HH:MM:SS.

O programa deverá seguir os seguintes passos:

1. Receber um **Unix Timestamp** que representa o horário de largada da corrida.
2. Receber um número N que representa a quantidade de competidores.
3. Receber N linhas, onde cada linha contém um **Unix Timestamp** que representa o horário de chegada de cada um dos N competidores.
4. Calcular o tempo de corrida do primeiro colocado e exibir no formato HH:MM:SS.

Entrada

A entrada será composta por múltiplas linhas:

1. A primeira linha conterá um **Unix Timestamp** que representa o horário de largada da corrida.
2. A segunda linha conterá um número inteiro N, que representa a quantidade de competidores.
3. As próximas N linhas conterão cada uma um **Unix Timestamp** que representa o horário de chegada de cada competidor.

Considere que o **Unix Timestamp** da largada será sempre menor que o **Unix Timestamp** da chegada de cada competidor e que o **Unix Timestamp** é sempre ≥ 0 . Além disso, considere $N \geq 2$.



Saída

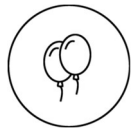
A saída deverá conter uma única linha com o tempo de corrida do primeiro colocado no formato HH:MM:SS. Considere que os dados de entrada possuem apenas um competidor vencedor.

Entrada	Saída
1694433600 5 1694434812 1694434727 1694434895 1694434452 1694434465	00:14:12
1694433600 8 1694437345 1694437280 1694437270 1694437331 1694437421 1694437389 1694437442 1694437305	01:01:10
98 3 100 101 99	00:00:01

No primeiro exemplo, a largada aconteceu no dia 11/09/2023 às 09:00:00, havia 5 competidores e o primeiro colocado levou 14 minutos e 12 segundos para completar a prova (1694434452). No segundo exemplo, a largada aconteceu no dia 11/09/2023 às 09:00:00, havia 8 competidores e o primeiro colocado levou 1 hora, 1 minuto e 10 segundos para completar a prova (1694437270).



<Página intencionalmente deixada em branco>



Simulação de Blockchain

Em uma rede blockchain, blocos de transações precisam ser validados por uma rede distribuída de nós antes de serem adicionados ao livro-razão distribuído. Cada bloco representa um conjunto de transações que deve ser validado para ser aceito na blockchain. Para ser validado, um bloco deve alcançar um consenso mínimo baseado em uma taxa de aprovação, ou seja, um percentual mínimo de "votos" favoráveis entre os nós participantes. Apenas blocos aprovados podem ser adicionados à cadeia, garantindo a segurança e a integridade do sistema.

Nesta simulação, sua tarefa é verificar uma série de blocos onde cada bloco tem:

1. Um número variável de votos de nós ("sim" ou "nao"), refletindo a aprovação ou rejeição por parte de cada nó da rede.
2. Uma taxa de aprovação mínima que define o percentual de votos "sim" necessário para validar o bloco.

Entrada

1. Uma lista de strings, onde cada string representa um bloco. Cada string possui o seguinte formato:

voto1, voto2, ..., votoN, taxaDeAprovacao

- Cada voto é uma String "sim" ou "nao", representando a posição de cada nó em relação ao bloco.
- A taxaDeAprovacao é um número decimal (entre 0 e 1) que representa a taxa mínima de aprovação necessária para que o bloco seja validado.

Regras para Validação

1. Taxa de Aprovação: Para um bloco ser aceito, o percentual de votos "sim" deve ser maior ou igual à taxa de aprovação mínima especificada para aquele bloco.
2. Rejeição de Blocos: Blocos que não atingem a taxa de aprovação mínima são rejeitados.

Saída

Ao final, exiba quais blocos foram aceitos e quais foram rejeitados.

Entrada	Saída
sim, sim, nao, sim, 0.75 sim, nao, nao, 0.5 sim, sim, nao, sim, sim, 0.6	Bloco 1: Bloco aceito Bloco 2: Bloco rejeitado Bloco 3: Bloco aceito Blocos aceitos: 1, 3



sim, sim, sim, sim, 0.9 sim, nao, sim, nao, 0.5 nao, nao, nao, sim, 0.25 sim, sim, nao, sim, nao, 0.7 sim, sim, nao, nao, 0.6 sim, sim, sim, nao, sim, sim, 0.8 nao, nao, sim, nao, nao, 0.2 sim, sim, nao, nao, nao, 0.4 sim, nao, sim, nao, sim, nao, nao, 0.5 sim, sim, sim, nao, sim, nao, sim, 0.65 sim, nao, nao, nao, nao, nao, 0.1 sim, sim, sim, nao, nao, sim, 0.6	Bloco 1: Bloco aceito Bloco 2: Bloco aceito Bloco 3: Bloco aceito Bloco 4: Bloco rejeitado Bloco 5: Bloco rejeitado Bloco 6: Bloco aceito Bloco 7: Bloco aceito Bloco 8: Bloco aceito Bloco 9: Bloco rejeitado Bloco 10: Bloco aceito Bloco 11: Bloco aceito Bloco 12: Bloco aceito Blocos aceitos: 1, 2, 3, 6, 7, 8, 10, 11, 12
nao, nao, sim, sim, 0.6	Bloco 1: Bloco rejeitado Blocos aceitos:



<Página intencionalmente deixada em branco>